

Chi-square test

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import chi2_contingency

observed = np.array([[60, 40], [45, 55]])

chi2, p_val, dof, expected = chi2_contingency(observed)

alpha = 0.05

print("\n Chi-Square Test Results:")

print(f"Chi-Square Statistic: {chi2:.4f}\nP-Value: {p_val:.4f}")

if p_val < alpha:

    result = " Null Hypothesis Rejected!\n Conclusion: Medicine's  effect different."

else:

    result = " Null Hypothesis Accepted!\n Conclusion: No significant difference."

print("\n" + result)


labels = ["Recovered", "Not Recovered"]

medicine_A = observed[0]

medicine_B = observed[1]

x = np.arange(len(labels))

width = 0.4

plt.bar(x - width/2, medicine_A, width, label=" A", color="lightblue")

plt.bar(x + width/2, medicine_B, width, label="B", color="purple")

plt.xticks(x, labels)

plt.ylabel("Number of Patients")

plt.title(" A vs B (Recovery Rates)")

plt.legend()
```

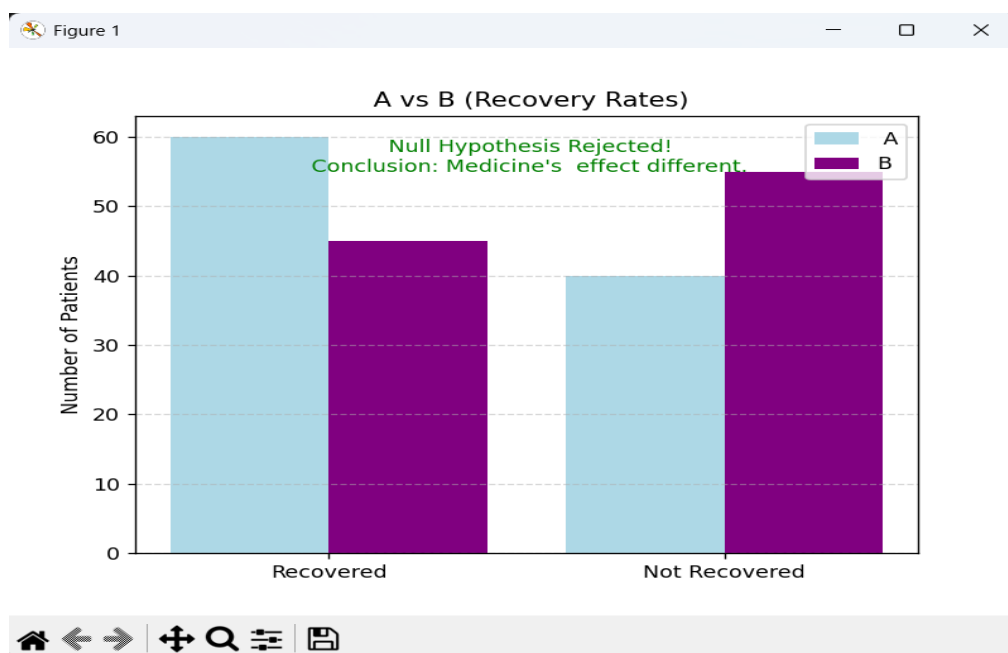
```
plt.grid(axis="y", linestyle="--", alpha=0.5)
```

```
plt.text(0.5, max(medicine_A.max(), medicine_B.max()) - 5, result, fontsize=10, color="green",  
ha="center")
```

```
plt.show()
```

OUTPUT:

```
c:\Users\Deepak Vishwakarma\Documents\SEM6\DS\practical4.  
  
Chi-Square Test Results:  
Chi-Square Statistic: 3.9298  
P-Value: 0.0474  
  
Null Hypothesis Rejected!  
Conclusion: Medicine's effect different.
```



PRACTICAL 5

Aim: ANOVA (Analysis of Variance)

- Perform one-way ANOVA to compare means across multiple groups.
- Conduct post-hoc tests to identify significant differences between group means.

CODE:

```
import scipy.stats as stats

import numpy as np

from statsmodels.stats.multicomp import pairwise_tukeyhsd


group1 = [23, 25, 28, 34, 30]
group2 = [19, 20, 22, 15, 23]
group3 = [18, 16, 20, 21, 17]
group4 = [28, 24, 26, 30, 29]


all_data = np.array(group1 + group2 + group3 + group4)


group_labels = (['Group1'] * len(group1) +
                 ['Group2'] * len(group2) +
                 ['Group3'] * len(group3) +
                 ['Group4'] * len(group4))


f_statistic, p_value = stats.f_oneway(group1, group2, group3, group4)


print("One-Way ANOVA Results:")
print(f"F-statistic: {f_statistic:.4f}")
```

```
print(f"P-value: {p_value:.4f}")
```

```
if p_value < 0.05:
```

```
    print("\n Significant difference found! Performing post-hoc analysis...")
```

```
    tukey_results = pairwise_tukeyhsd(all_data, group_labels)
```

```
    print("\n Tukey-Kramer Post-Hoc Test Results:")
```

```
    print(tukey_results)
```

```
else:
```

```
    print("\n No significant difference found between groups.")
```

OUTPUT:

```
[Running] python -u "c:\Users\Deepak Vishwakarma\Documents\SEM6
One-Way ANOVA Results:
F-statistic: 13.0757
P-value: 0.0001

Significant difference found! Performing post-hoc analysis...

Tukey-Kramer Post-Hoc Test Results:
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====
group1 group2 meandiff p-adj  lower  upper reject
-----
Group1 Group2      -8.2 0.0035 -13.7991 -2.6009  True
Group1 Group3      -9.6 0.0008 -15.1991 -4.0009  True
Group1 Group4       -0.6 0.9896  -6.1991  4.9991 False
Group2 Group3       -1.4 0.8895  -6.9991  4.1991 False
Group2 Group4        7.6 0.0065   2.0009 13.1991  True
Group3 Group4        9.0 0.0015   3.4009 14.5991  True
-----

[Done] exited with code=0 in 3.789 seconds
```

PRACTICAL 6

AIM: Regression and Its Types

- Implement simple linear regression using a dataset.
- Explore and interpret the regression model coefficients and goodness-of-fit measures.
- Extend the analysis to multiple linear regression and assess the impact of additional predictors.

1.Implement simple linear regression using a dataset.

2.Explore and interpret the regression model coefficients and goodness-of-fit Measures.

CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

from sklearn.datasets import fetch_california_housing


housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)


X = df[['AveRooms']]

y = housing.target


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


model = LinearRegression()

model.fit(X_train, y_train)


y_pred = model.predict(X_test)


print(f" Mean Squared Error: {mean_squared_error(y_test, y_pred):.4f}")

print(f" R-Squared Score: {r2_score(y_test, y_pred):.4f}")
```

```

print(f" Intercept: {model.intercept_:.4f}")

print(f" Coefficient: {model.coef_[0]:.4f}")


plt.scatter(X_test, y_test, color="purple", label="Actual Prices")

plt.plot(X_test, y_pred, color="green", linewidth=2, label="Regression Line")

plt.xlabel("Average Rooms")

plt.ylabel("House Price")

plt.title("Simple Linear Regression")

plt.legend()

plt.show()

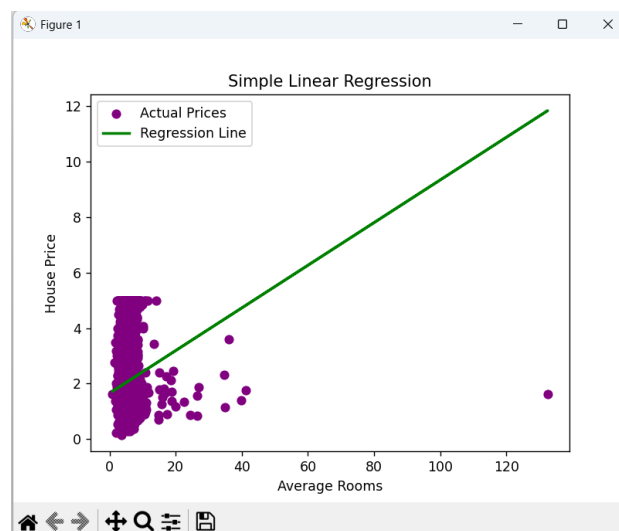
```

OUTPUT:

```

[Running] python -u "c:\Users\Deepak Vishwakarma\Docum
Mean Squared Error: 1.2923
R-Squared Score: 0.0138
Intercept: 1.6548
Coefficient: 0.0768

```



3.Extend the analysis to multiple linear regression and assess the impact of additional predictors.

CODE:

```

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

```

```
from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

from sklearn.datasets import fetch_california_housing


housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)


X = df[['AveRooms', 'HouseAge', 'Population']]

y = housing.target


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


model = LinearRegression()

model.fit(X_train, y_train)


y_pred = model.predict(X_test)


print(f" Mean Squared Error: {mean_squared_error(y_test, y_pred):.4f}")

print(f" R-Squared Score: {r2_score(y_test, y_pred):.4f}")

print(f" Intercept: {model.intercept_: .4f}")

print(f" Coefficients: {model.coef_}")


plt.figure(figsize=(9,5))

plt.bar(X.columns, model.coef_, color='blue')

plt.xlabel("Features")

plt.ylabel("Coefficient Value")

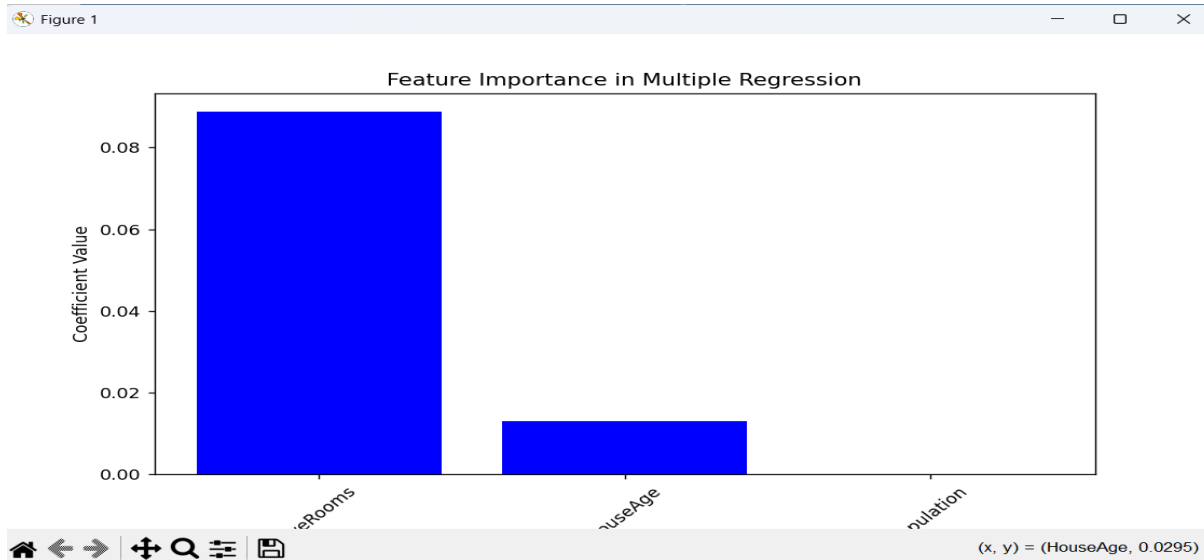
plt.title("Feature Importance in Multiple Regression")

plt.xticks(rotation=45)

plt.show()
```

OUTPUT:

```
[Running] python -u "c:\Users\Deepak Vishwakarma\Documents\SEM6  
Mean Squared Error: 1.2697  
R-Squared Score: 0.0311  
Intercept: 1.1754  
Coefficients: [8.88258229e-02 1.29967829e-02 2.93829862e-05]
```



PRACTICAL 7

AIM: Logistic Regression and Decision Tree.

- Build a logistic regression model to predict a binary outcome.
- Evaluate the model's performance using classification metrics (e.g., accuracy, precision, recall).
- Construct a decision tree model and interpret the decision rules for classification.

CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, precision_score, recall_score


data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)

y = data.target


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

```
log_model = LogisticRegression(max_iter=5000)
```

```
log_model.fit(X_train_scaled, y_train)
```

```
y_pred_log = log_model.predict(X_test_scaled)
```

```
tree_model = DecisionTreeClassifier(max_depth=3, random_state=42)
```

```
tree_model.fit(X_train, y_train)
```

```
y_pred_tree = tree_model.predict(X_test)
```

```
def evaluate_model(model_name, y_test, y_pred):
```

```
    print(f" {model_name} Performance:")
```

```
    print(f" Accuracy : {accuracy_score(y_test, y_pred):.4f}")
```

```
    print(f" Precision: {precision_score(y_test, y_pred, zero_division=0):.4f}")
```

```
    print(f" Recall   : {recall_score(y_test, y_pred, zero_division=0):.4f}")
```

```
    print("-" * 35)
```

```
evaluate_model("Logistic Regression", y_test, y_pred_log)
```

```
evaluate_model("Decision Tree", y_test, y_pred_tree)
```

```
plt.figure(figsize=(15, 8))
```

```
plot_tree(
```

```
    tree_model,
```

```
    feature_names=data.feature_names,
```

```
    class_names=data.target_names,
```

```
    filled=True
```

)

```
plt.title("Decision Tree Visualization")
```

```
plt.tight_layout()
```

```
plt.show()
```

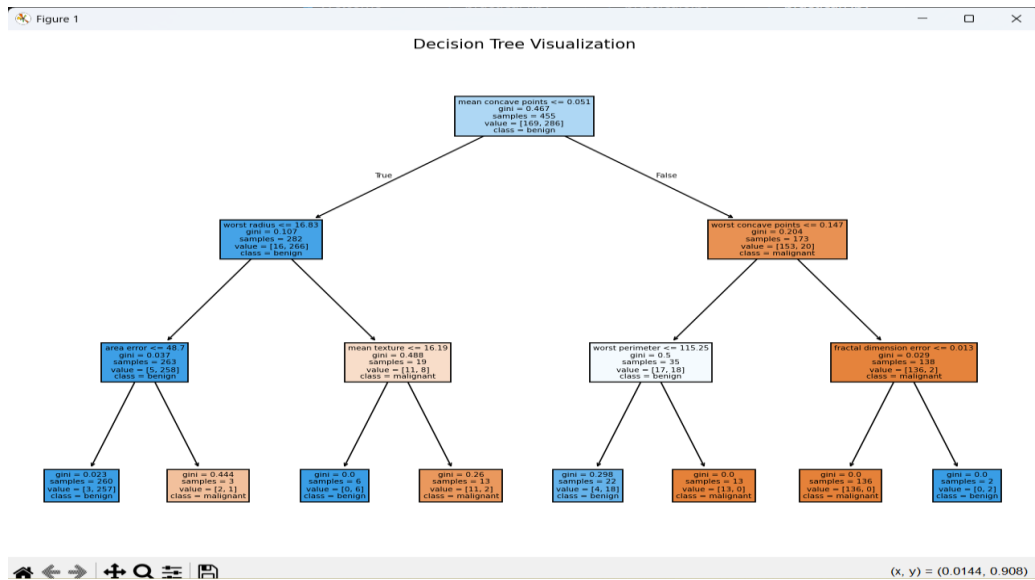
OUTPUT:

```
[Running] python -u "c:\Users\Deepak Vishwakarma\Documents\
Logistic Regression Performance:
Accuracy : 0.9737
Precision: 0.9722
Recall   : 0.9859

-----

Decision Tree Performance:
Accuracy : 0.9474
Precision: 0.9452
Recall   : 0.9718

-----
```



PRACTICAL 8

AIM: K-Means Clustering

- Apply the K-Means algorithm to group similar data points into clusters.
- Determine the optimal number of clusters using elbow method or silhouette analysis.
- Visualize the clustering results and analyze the cluster characteristics.

CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

from sklearn.datasets import make_blobs

from sklearn.preprocessing import StandardScaler


X, _ = make_blobs(n_samples=300, centers=4, cluster_std=1.0, random_state=42)

scaler = StandardScaler()

X = scaler.fit_transform(X)

wcss = []


for k in range(1, 11):

    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)

    kmeans.fit(X)

    wcss.append(kmeans.inertia_)


print("\nWCSS Values for Different K:")

for i, val in enumerate(wcss, start=1):

    print(f"K={i}: WCSS={val:.2f}")


plt.figure(figsize=(8, 5))

plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
```

```
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.title('Elbow Method for Optimal K')
plt.grid(True)
plt.show()

optimal_k = 4

kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X)

print("\nCluster Assignments for First 10 Points:")
for i in range(10):
    print(f"Point {i+1}: Cluster {y_kmeans[i]}")

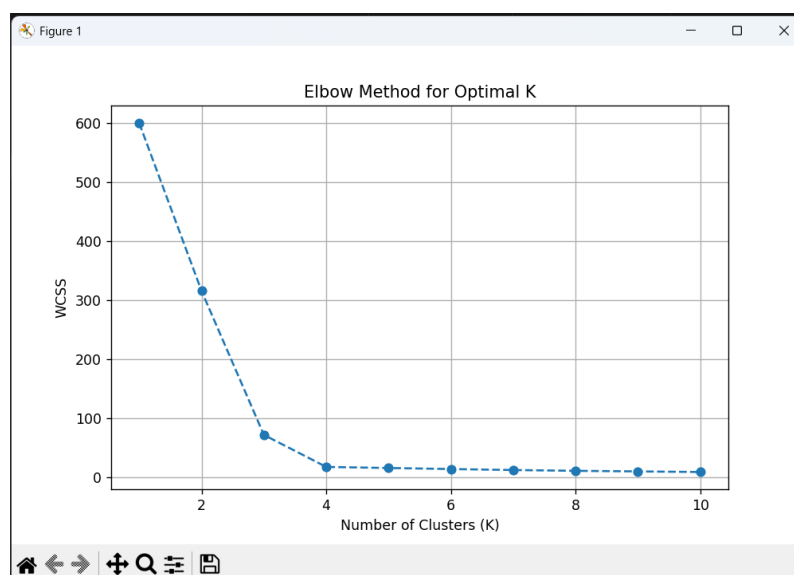
print("\nCluster Centroids:")
print(kmeans.cluster_centers_)

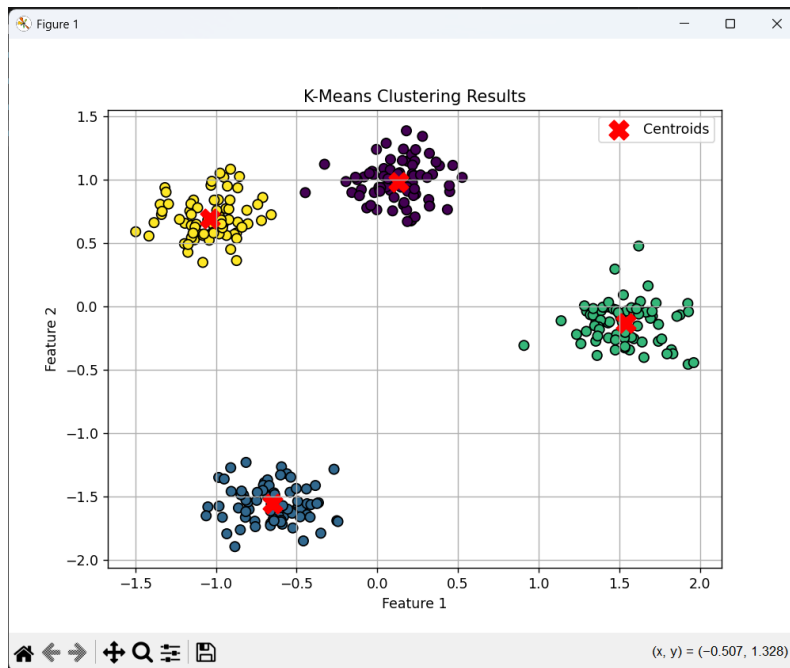
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap='viridis', edgecolors='k', s=50)
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    s=200,
    c='red',
    marker='X',
    label='Centroids'
)
plt.xlabel('Feature 1')
```

```
plt.ylabel('Feature 2')  
plt.title('K-Means Clustering Results')  
plt.legend()  
plt.grid(True)  
plt.show()
```

OUTPUT:

```
[Running] python -u "c:\Users\Deepak Vishwak  
WCSS Values for Different K:  
K=1: WCSS=600.00  
K=2: WCSS=315.49  
K=3: WCSS=71.69  
K=4: WCSS=17.47  
K=5: WCSS=15.64  
K=6: WCSS=13.85  
K=7: WCSS=12.18  
K=8: WCSS=10.84  
K=9: WCSS=9.86  
K=10: WCSS=8.83  
  
Cluster Assignments for First 10 Points:  
Point 1: Cluster 3  
Point 2: Cluster 3  
Point 3: Cluster 0  
Point 4: Cluster 1  
Point 5: Cluster 3  
Point 6: Cluster 1  
Point 7: Cluster 2  
Point 8: Cluster 1  
Point 9: Cluster 0  
Point 10: Cluster 2  
  
Cluster Centroids:  
[[ 0.13143596  0.98564007]  
 [-0.64929923 -1.55675468]  
 [ 1.53816887 -0.12928168]  
 [-1.03586967  0.69654164]]  
[Done] exited with code=0 in 69.575 seconds
```





PRACTICAL 9

AIM: Principal Component Analysis (PCA)

- Perform PCA on a dataset to reduce dimensionality.
- Evaluate the explained variance and select the appropriate number of principal components
- Visualize the data in the reduced-dimensional space.

CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA


iris = load_iris()

iris_df = pd.DataFrame(data=np.c_[iris['data'], iris['target']], columns=iris['feature_names'] +
['target'])


X = iris_df.drop('target', axis=1)

y = iris_df['target']


scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


pca = PCA()

X_pca = pca.fit_transform(X_scaled)


explained_variance_ratio = pca.explained_variance_ratio_


plt.figure(figsize=(8,6))

plt.plot(np.cumsum(explained_variance_ratio), marker='o', linestyle='--')

plt.title('Explained Variance Ratio')
```



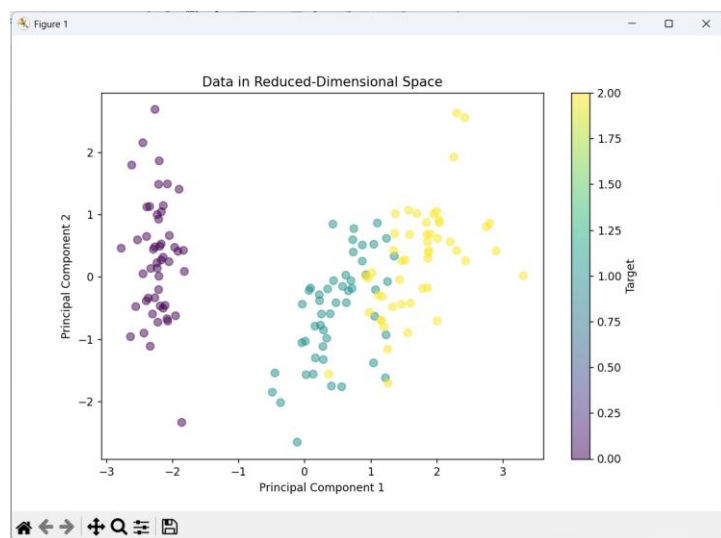
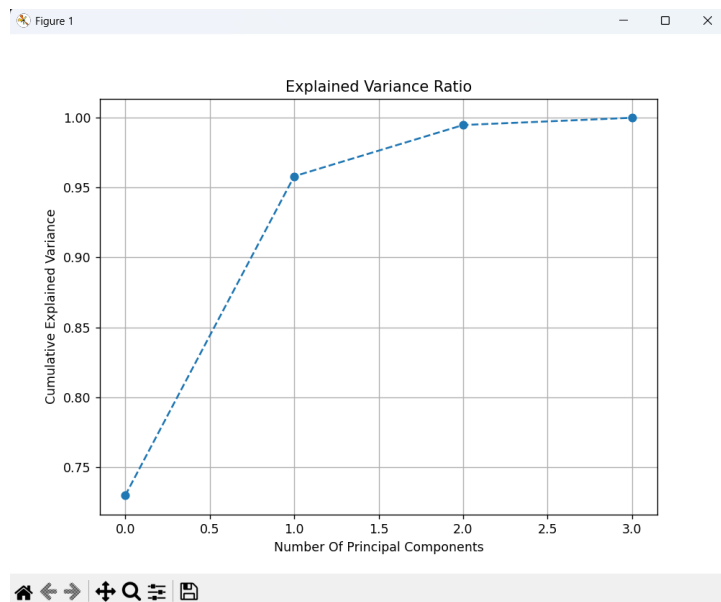
```
plt.xlabel('Number Of Principal Components')
plt.ylabel('Cumulative Explained Variance')
plt.grid(True)
plt.show()

cumulative_variance_ratio = np.cumsum(explained_variance_ratio)
n_components = np.argmax(cumulative_variance_ratio >= 0.95) + 1
print(f'Number of principal components to explain 95% variance: {n_components}')

pca = PCA(n_components=n_components)
X_reduced = pca.fit_transform(X_scaled)

plt.figure(figsize=(9,6))
plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=y, cmap='viridis', s=50, alpha=0.5)
plt.title('Data in Reduced-Dimensional Space')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar(label='Target')
plt.show()
```

OUTPUT:



PRACTICAL 10

AIM: Data Visualization and Storytelling.

- Create meaningful visualizations using data visualization tools
- Combine multiple visualizations to tell a compelling data story.
- Present the findings and insights in a clear and concise manner.

CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

data = {

    'Category':["Action", "Comedy", "Drama", "Horror", "Romance", "Sci-Fi"],

    'Views':[120, 150, 90, 60, 80, 110],

    'Rating':[4.5, 4.0, 4.8, 3.5, 4.2, 4.6]

}

df = pd.DataFrame(data)

print("DataFrame:")

print(df)

fig,axes = plt.subplots(1, 2, figsize=(12, 5))

sns.barplot(x='Category', y='Views', data=df, ax=axes[0], palette='viridis')

axes[0].set_title('Views by Category')

sns.scatterplot(x='Views', y='Rating', data=df, ax=axes[1], hue='Category', s=100)

axes[1].set_title('Rating vs Views')

plt.tight_layout()

plt.show()
```

OUTPUT:

DataFrame:

	Category	Views	Rating
0	Action	120	4.5
1	Comedy	150	4.0
2	Drama	90	4.8
3	Horror	60	3.5
4	Romance	80	4.2
5	Sci-Fi	110	4.6

